

Designed by Remus Teodorescu 2026

LECTURE 6.2

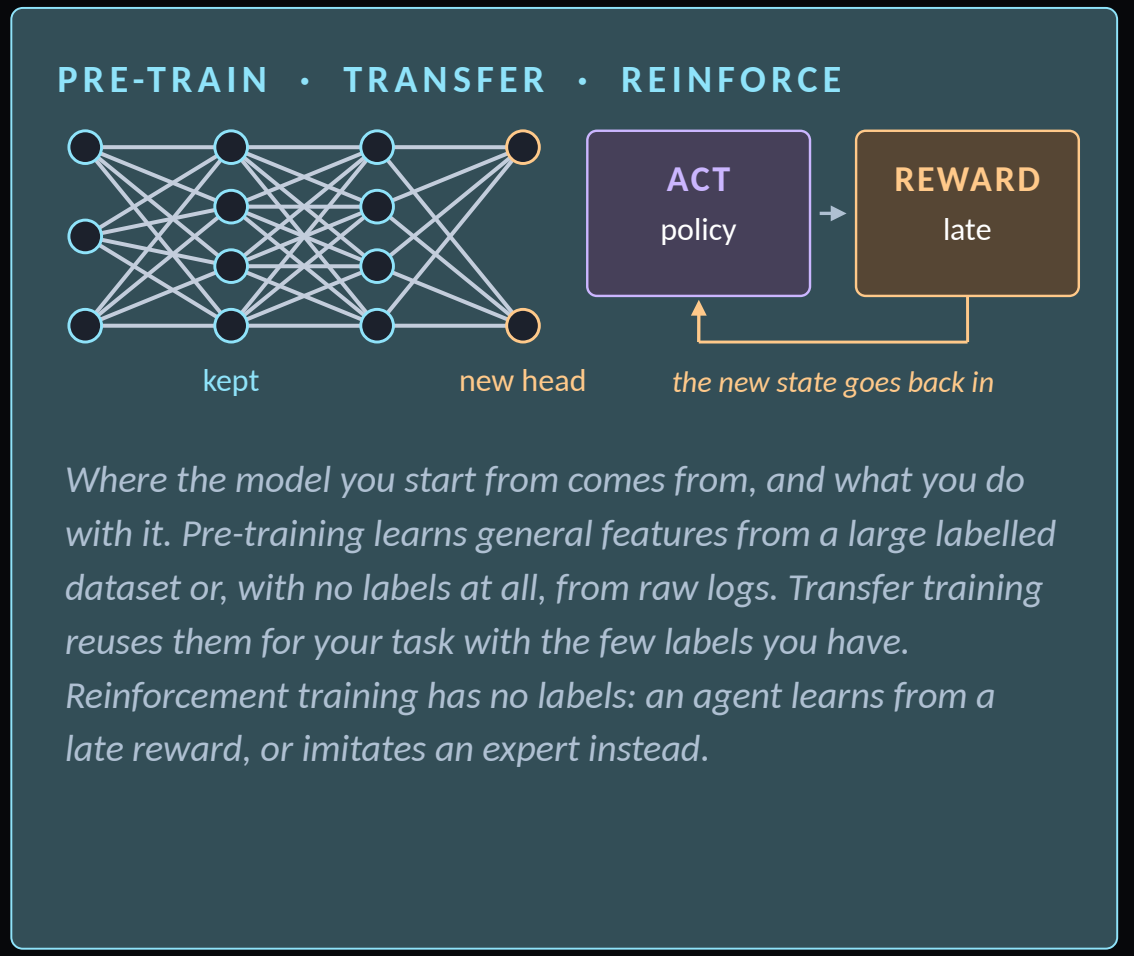
Training Philosophies

Remus Teodorescu

With support from Research Assistant Noman Khan - nomank@energy.aau.dk

Aalborg University · ret@et.aau.dk

Deep Learning for Engineering · 2026



Where the model you start from comes from, and what you do with it. Pre-training learns general features from a large labelled dataset or, with no labels at all, from raw logs. Transfer training reuses them for your task with the few labels you have. Reinforcement training has no labels: an agent learns from a late reward, or imitates an expert instead.

Learning Objectives

SECTIONS, IN ORDER

1. **Pre-training** — supervised on labelled data, self-supervised on raw data
2. **Transfer training** — transfer, fine-tuning, and where they are used
3. **Reinforcement training** — reinforcement and imitation, and where they are used

Deployment on the car, quantisation, pruning and compilation, is taught in L11.1, where the network runs in a real-time control loop.

READ, AND WHAT EACH ANSWERS

UDL ch. 9.3.7	where does the model you start from come from?
UDL ch. 9.3.6	how do you reuse it with the few labels you have?
UDL ch. 19	how do you learn to act from a late reward?

Training Philosophies

Training philosophy — where the training signal comes from: a label for every input (supervised), a task made from the data itself (self-supervised), a model already trained on another task (transfer), an expert's actions (imitation), or a reward that arrives late (reinforcement).

FIVE SOURCES OF A TRAINING SIGNAL

	the signal	an energy example
supervised	a label for every input	a load forecast
self-supervised	a task made from the data	battery logs, no labels
transfer	a model trained elsewhere	NMC cells to LFP
imitation	an expert's actions	approximate MPC
reinforcement	a late reward	battery dispatch

Self-supervision makes the model you start from; transfer, imitation and reinforcement are what you do to it afterwards.

PRE-TRAIN, THEN POST-TRAIN



IN THIS LECTURE

section 1 pre-training: labelled, or not

section 2 transfer training: reuse, fine-tune

section 3 reinforcement training: reward, imitation

Supervised Pre-Training

Supervised pre-training — training a network on a large labelled dataset for a related task, so that its layers learn general features before any of your own labels are used: a million labelled photographs for images, years of labelled lab cycling for a battery-health model. The network it produces is the starting point for transfer.

THE STARTING MODEL

- **the source** — a large, labelled, related dataset
- **what it gives** — general features in the early layers
- **the cost** — somebody labels it; often somebody else did

Pre-trained networks are published in model zoos: torchvision's ResNet-18, trained on ImageNet, is the one the camera car fine-tunes.

LABELLED, OR NOT

	supervised	self-supervised
labels	many, for a related task	none
data	curated	plentiful, raw
battery	NMC, 10 000 cycles	fleet logs
images	ImageNet	photographs, masked
then	transfer training	transfer training

Both produce a network whose early layers already read the data. Section 2 reuses it; which of the two made it matters less than how close its data was to yours.

Self-Supervised Pre-Training

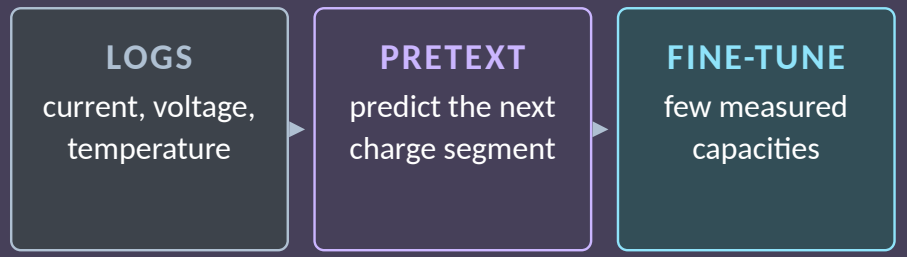
Self-supervised pre-training — also called unsupervised pre-training: training on plenty of unlabelled data with a pretext task whose answer is in the data itself, such as predicting a hidden part of a signal or its next segment. It needs no labels, and the features it learns are what fine-tuning reuses.

BEFORE POST-TRAINING, AND AS PART OF IT

- **the prerequisite** — post-training needs a pre-trained model
- **continued pre-training** — more self-supervision on your data
- **why it pays** — battery logs look nothing like general data

Continued pre-training, also called domain-adaptive pre-training, is for engineering often the most valuable post-training step.

BATTERY AGEING, FEW LABELS



THE SAME SHAPE ELSEWHERE

PMU streams	forecast the next window, then detect faults
vibration	reconstruct, then diagnose bearing faults
EV fleets	mask part of a cycle, then estimate health

Transfer Learning

Transfer learning — training a network for a new task starting from the weights of a network trained on a related task with far more data. The early layers, which have learned general features such as edges and textures, are kept; the last layers are replaced and trained on the new task, so a few hundred labelled examples can be enough.

WHAT IS KEPT, WHAT IS REPLACED

- **keep** — the early layers, which learned edges and textures
- **replace** — the head, which named the old task's classes
- **the saving** — a few hundred labels, not a hundred thousand

TALK · 2 MIN How do you understand transfer learning, in your own words? Then, in the stack on the right, which rows would you keep for a fault classifier on a new machine, and why?

WHAT EACH DEPTH HAS LEARNED

layer 1	oriented edges
layer 2	corners, colour blobs
layers 3 – 4	textures, repeated motifs
layers 5 – 6	object parts
the head	the thousand class names

Only the last row is about the original task. That is the row you throw away and replace with your own — two classes, or one steering angle.

Transfer Learning in Engineering

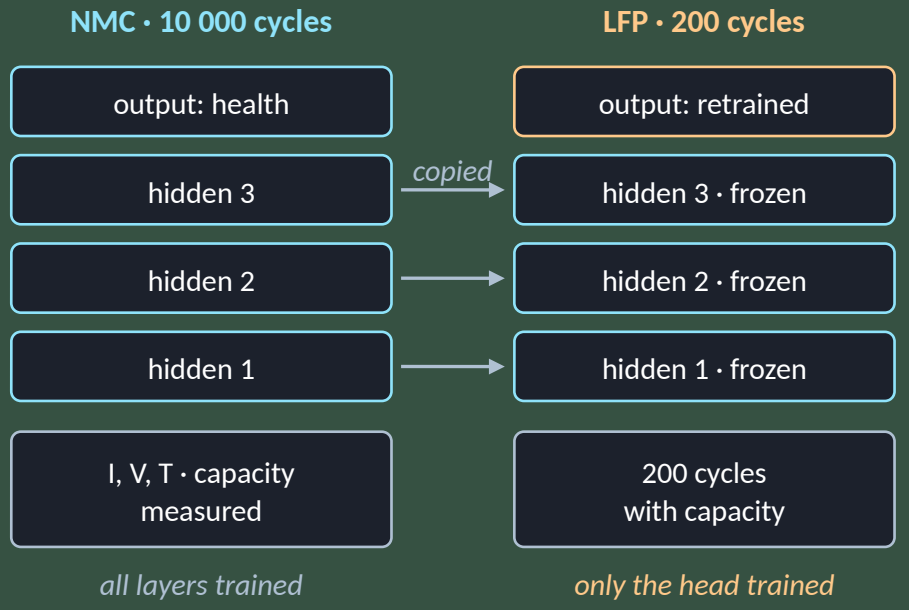
Battery chemistry transfer — the standard engineering case. A state-of-health model trained on a large dataset from one cell type, NMC cells with years of lab cycling, is reused for a new LFP cell with only a few cycles: the early layers, generic features of voltage and current curves, are kept, and the last layer is retrained on the small set.

OTHER CASES WITH THE SAME SHAPE

- **another grid** — a power-flow or stability model, moved
- **another unit** — fault diagnosis, one converter to the next
- **simulation to real** — pre-train on cheap simulated data

Transfer starts from a labelled task elsewhere; self-supervision from a pretext task on unlabelled data. In practice the two are combined.

NMC TO LFP: COPY, FREEZE, RETRAIN



The frozen layers keep what the NMC cells taught them; two hundred LFP cycles are enough for the head.

Fine-Tuning

Fine-tuning — continuing to train a pre-trained network on the new task, usually at a much smaller learning rate. One can train only the new head, unfreeze the last blocks as well, or retrain everything, depending on how much labelled data there is and how different the new task is.

THREE SETTINGS, AND HOW TO CHOOSE

- **head only** — the least data, and the first try
- **the last blocks too** — at a tenth of the learning rate
- **Ex_06 notebook 03** — both routes beat scratch on thirty labels

FINE-TUNING IN PYTORCH

```
model = torchvision.models.resnet18(weights='DEFAULT')  # trained on ImageNet
for p in model.parameters(): p.requires_grad = False    # freeze the body
model.fc = nn.Linear(512, 3)                            # a new head: 3 classes
opt = torch.optim.Adam(model.fc.parameters(), lr=1e-3)  # train the head
# then unfreeze model.layer4 at lr=1e-4                 # a tenth of the rate
```

WHAT TO UNFREEZE

setting	data	domain
head only	small	similar
last blocks	medium	the usual answer
everything	large	different
learning rate	10 – 100 × smaller	always

CATASTROPHIC FORGETTING

Fine-tune too hard and the network loses the general features it was carrying. The cure is the small learning rate, and freezing the early layers.

Reinforcement Learning

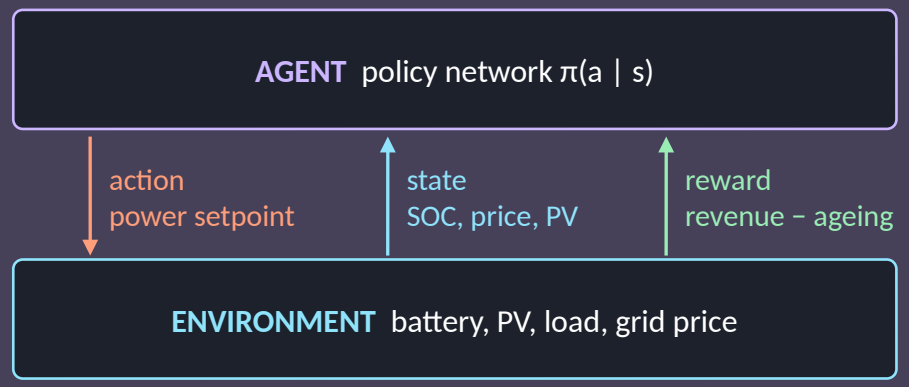
Reinforcement learning — learning to act by trial and error. An agent observes the state of an environment, takes an action chosen by its policy, and receives a reward; the goal is a policy that maximises the expected sum of future rewards, the return. No correct action is ever given.

AGAINST SUPERVISED LEARNING, AND AGAINST CONTROL

	supervised	reinforcement	control
the signal	a target per input	a scalar, late	a cost per step
the model	of the mapping	of the policy	of the plant
the world	not needed	a simulator, or samples	a plant model
guarantees	held-out error	none	stability proofs
vocabulary	x, y	s, a, π, r, G	x, u, controller, cost

TALK · 2 MIN How would you explain reinforcement learning to a control engineer? Then, in the loop on the right, which box plays the controller, and what plays the cost?

A BATTERY IN A MICROGRID



$a \sim \pi_\theta(a \mid s)$ *s* the state, *a* the action

ONE EPISODE

```
state = env.reset()                # the first state
for t in range(400):               # one episode
    action = policy(state).sample()  # a ~ π(a|s)
    state, r, done = env.step(action)  # the world
    if done: break                 # a crash, or the end
```

Reinforcement Learning in Engineering

Where it pays off — a sequential problem, an objective that is delayed or hard to write as a loss, and a simulator or digital twin to train in. The usual pattern: train in the simulator, then deploy behind a safety layer that clips the actions.

STORAGE, GRID AND POWER ELECTRONICS

- **storage** — dispatch against price, fast charging against ageing
- **grid** — frequency, voltage, emergency load shedding
- **converters, HVAC, wind farms** — gains, cooling, wake steering

WHERE IT DOES NOT FIT

A problem already solved as an optimisation with a known model (MPC, unit commitment); a one-shot prediction (state of health is regression); a safety-critical loop with no trustworthy simulator.

ONE STEP, EVERY 15 MINUTES

observe	SOC 62 %, 0.41 €/kWh, PV 0, load 30 kW
act	discharge at 20 kW
reward	earned 2.05 €, ageing cost 0.30 €
learn	make high-return actions more likely

Illustrative numbers: 20 kW for 15 minutes is 5 kWh, and at 0.41 €/kWh that earns 2.05 €.

Return and Value

Return — the sum of the rewards from now on, each discounted by a factor γ per step, so that near rewards count more than distant ones. The value of a state is the return expected from it under the policy; the action value also fixes the first action.

THE OBJECTIVE IS A SUM OVER THE FUTURE

- **the return** — the sum over the future, not the next reward
- **read γ as a time constant** — set it from your horizon

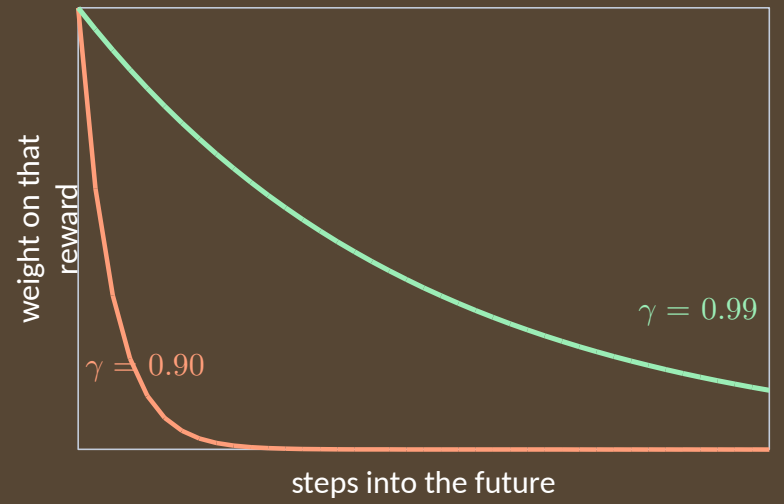
$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

G_t the return from step t, r one reward, $\gamma \in (0, 1)$ how far ahead you look

THE RETURN IN PYTHON

```
G, returns = 0.0, []            # start at the end of the episode
for r in reversed(rewards):      # the last reward first
    G = r + gamma * G            # G_t = r_t + γ G_t+1
    returns.insert(0, G)         # one return per step
```

HOW FAR AHEAD γ LOOKS



$$Q^{\pi}(s, a) = \mathbb{E}[G_t \mid s_t = s,\ a_t = a]$$

$\gamma = 0.90$ is effectively blind beyond fifty steps. Choose it from your system's timescale.

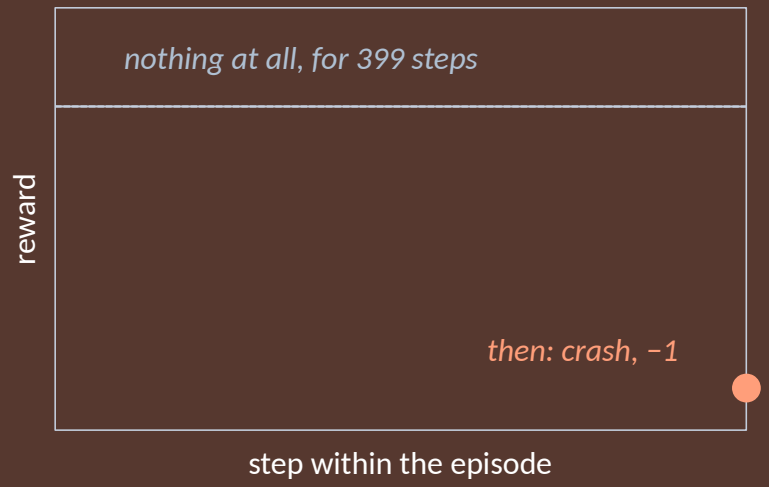
Credit Assignment

Credit assignment — the problem of deciding which of many earlier actions caused a reward that arrives late. With one reward at the end of a long episode every action is a suspect, and it takes many episodes to tell a good action from a lucky one.

ONE LATE NUMBER, FOUR HUNDRED DECISIONS

- **an episode** — four hundred steps, one number at the end
- **the blunt approach** — credit every action, many episodes
- **the refinements** — discounting, learned values, a baseline
- **shaping the reward** — helps, and can teach the wrong task

ONE REWARD, FOUR HUNDRED ACTIONS



A sparse reward, and it is the normal case. Shaping it — small rewards along the way — helps a great deal and is also how you accidentally teach the wrong task.

Policy Gradient

Policy gradient — a method that trains a stochastic policy network directly: run episodes, then raise the log-probability of each action taken in proportion to the return that followed. Only the policy needs to be differentiable, not the world.

A GRADIENT WITHOUT DIFFERENTIATING THE WORLD

- **make the policy stochastic** — a distribution over actions
- **run an episode** — record what was done, and what followed
- **raise what worked** — in proportion to the return

THE POLICY GRADIENT IN PYTORCH

```
dist = Categorical(logits=policy(states))     # π(a|s) at every step
logp = dist.log_prob(actions)                 # of what was done
loss = -(logp * returns).mean()               # likelihood, weighted by G
opt.zero_grad(); loss.backward(); opt.step()  # one policy update
```

THE POLICY GRADIENT

$$\nabla_\theta J = \mathbb{E}[\nabla_\theta \log \pi_\theta(a \mid s)\, G_t]$$

J the expected return, G_t the return from step t, π_θ the policy

Raise the probability of what worked, in proportion to how well it worked: the maximum likelihood recipe with the outcome as a weight.

WHAT IT COSTS

variance	enormous — hence the baselines
samples	millions of episodes is normal
stability	no guarantee of any kind

Imitation Learning

Imitation learning — learning a policy by copying an expert: record what the expert observed and what they did, and fit a network to predict the action from the observation. It is supervised learning with no reward, and it cannot do better than the expert.

COPY THE EXPERT, THEN REPLACE IT

- **a regression** — the target is the expert's action
- **no reward, no exploring** — safe on real hardware
- **compounding error** — label the states it reaches

$$\hat{\theta} = \operatorname{argmin}_\theta \sum_i (\pi_\theta(s_i) - a_i)^2$$

$\hat{\theta}$ the fitted policy, (s_i, a_i) one state and the action the expert took, no reward anywhere

BEHAVIOURAL CLONING IN PYTORCH

```
pred = model(states)                          # the action it would choose
loss = nn.MSELoss()(pred, expert_actions)     # squared error to the expert's
opt.zero_grad(); loss.backward(); opt.step()  # an ordinary supervised step
```

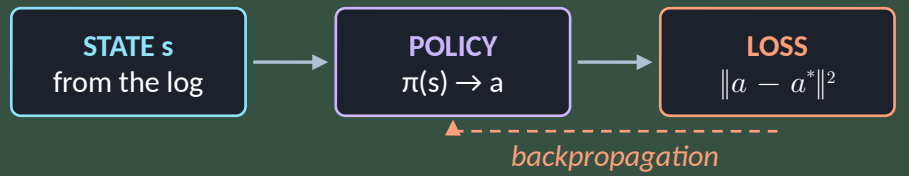
1 · COLLECT DEMONSTRATIONS

state s to the expert, action a* back; every pair logged



2 · TRAIN THE POLICY BY SUPERVISED LEARNING

input the state, target the expert's action



3 · DEPLOY: THE NETWORK REPLACES THE EXPERT

state s in, action a out, in microseconds



DAgger: the expert labels the new states; back to 2.

Imitation Learning in Engineering

Approximate MPC — the flagship engineering use: run a model-predictive controller offline over many scenarios, log its states and actions, and train a network to reproduce them. The network runs in microseconds with close to MPC performance, where the optimiser is too slow or too heavy for the embedded controller.

WHERE AN EXPERT IS TOO SLOW OR TOO COSTLY

- **fast charging** — an electrochemical optimum, onto the BMS
- **grid operation** — imitate optimal power flow, in milliseconds
- **wind farms, HVAC** — replace a slow supervisory optimiser
- **warm-starting RL** — begin from a working controller

IMITATE OR REINFORCE

	imitation	reinforcement
needs	an expert	a reward, a simulator
data	logged (s, a*)	many episodes
better than the expert	no	it can be
on hardware	safe	explores

THE COMMON THREAD

An expert exists, a solver, an optimiser on a simulator or a skilled operator, but is too slow or too costly to run where the decision is needed.

Advanced Topics

RELATED TOPICS LEFT OUT, AND WHERE THEY MATTER

topic	what it adds	read	where it makes a difference
multi-task learning	one network, several related tasks	UDL ch. 9.3.6	shared features, less data
BERT and GPT	self-supervision at enormous scale	UDL ch. 12.6 – 12.7	language models
fitted Q-learning	a network learns action values	UDL ch. 19.4	Atari games, robotics
actor-critic	a policy and a value, together	UDL ch. 19.6	continuous control, robots
offline RL	learns from logged data alone	UDL ch. 19.7	driving, trading, medicine
RL from human feedback	a reward model from preferences	UDL ch. 19 notes	chatbots such as ChatGPT

WHY THEY ARE LEFT OUT

Each refines today's two ideas, and most pay off on very large data or in simulation. Deployment on the car, quantisation, pruning and compilation, is taught in L11.1.

EXERCISE

Exercise 6

THE NOTEBOOKS, IN ORDER

- **00 to 02** — L6.1's: the setup, the loss recipe, Adam then L-BFGS
- **03** — a second machine, ten labels a class: scratch, probe and fine-tune compared
- **04** — a battery trades by REINFORCE against the exact optimum, then imitates it
- **05** — the report: every notebook's answers, gathered, and one question to conclude

THE QUESTION EACH NOTEBOOK ANSWERS

03	which start wins with thirty labels?
04	how close does RL get to the optimum?

REPORT QUESTION, TO CONCLUDE

Across the notebooks: which one choice — the loss, the optimiser or the starting weights — changed your result most, and what number shows it?

SUMMARY

Key Takeaways

PRE-TRAINING AND TRANSFER

what it is	start from weights trained on a bigger, related task
function	keep the general features; a new head for your task
train, tune	head first, then the last blocks, at a tenth of the rate
vs scratch	wins on few labels; scratch catches up as labels grow
used for	battery chemistries, new grids, simulation to real

REINFORCEMENT TRAINING

what it is	learning to act from a late scalar reward
function	a policy: state in, action out, maximising the return
train, tune	policy gradient; γ from the horizon; many episodes
vs supervised	no target per input: noisy, hungry, no guarantee
used for	dispatch, charging, converters; imitate an MPC

Next — L7.1: the same optimisers, pointed at a physics residual.

Questions

1 – 2 · PRE-TRAINING AND TRANSFER

1 **Why can a network trained on another task help with yours?**

Its first layers hold general features, such as edges and textures, that carry over. They are kept, and only a new last layer is trained on your data. With few labels this starts far ahead of random weights. *Go deeper: transfer with few labels (Ex_06 notebook 03); UDL ch. 9.3.6.*

2 **How is a transferred network fine-tuned, and when does transfer stop helping?**

Train the new head first, then unfreeze the last blocks at about a tenth of the learning rate, so that what the network knew is not overwritten. As the labels grow, training from scratch catches up. *Go deeper: the error against the number of labels (Ex_06 notebook 03); UDL ch. 9.3.7.*

3 – 4 · REINFORCEMENT TRAINING

3 **What does a network learn in reinforcement learning, and from what?**

A policy: a map from the state to an action, learned from a reward that arrives late. It is trained to make the sum of future rewards as large as possible. In control terms: state, input, controller and cost. *Go deeper: state, action and reward (UDL ch. 19.1 – 19.2); the discount factor (UDL ch. 19.2).*

4 **When is imitating an expert a better choice than reinforcement learning?**

Reinforcement learning has no target for each input, only a late reward, so training is noisy, needs many episodes and gives no guarantee. If an expert such as an MPC controller exists, imitating it is ordinary supervised learning, and much cheaper. *Go deeper: imitation learning (UDL ch. 19.5 – 19.7); which of the expert's errors the copy inherits.*

NEXT

L7.1 — Physics-Informed Neural Networks

Part 1 ends here. The optimisers of L6.1 are about to be pointed at a physics residual: the same Adam, the same L-BFGS, and no data in the loss at all.

BEFORE THE NEXT SESSION

run	Ex_06 notebooks 03 and 04
read	UDL ch. 9.3.6 – 9.3.7, 19.1 – 19.5
bring	your notebook 03 comparison
think about	what your physics residual would be

Six blocks of general deep learning are finished. From L7 every network you build has an equation in its loss.